Segundo trabajo de evaluación continua: estimación de la estructura temporal de los tipos de interés cupón cero

Luiz Alberto Luzuriaga Izquierdo y Andrés Otárola
Facultad de ciencias económicas, empresariales y turismo.
Universidad de Alcalá de Henares.
Curso 2026-2027

Resumen

A partir de las cotizaciones de bonos y obligaciones del Estado del 11 de septiembre de 2026 (fecha valor 14 de septiembre) se estima la ETTI cupón cero con el modelo de Svensson y, como contraste, con splines cúbicas sobre la función de descuento. Se utilizan 57 títulos (precio ex-cupón medio compra/venta) y se minimizan los errores de precio ponderados por el inverso de la duración. El modelo de Svensson ajusta con un error cuadrático medio de 3,9 puntos básicos en TIR y la spline con 2,8. La curva es creciente, del 2,62 % a 0,1 años al 4,70 % a 45 años.

Palabras clave: estructura temporal de los tipos de interés, Svensson, splines cúbicas, tipos cupón cero, bonos del Estado.

Abstract

Using the Spanish Treasury bond quotes of 11 September 2026 (value date 14 September), the zero-coupon term structure is estimated with the Svensson model and, as a robustness check, with cubic splines on the discount function. 57 bonds are used (mean bid/ask clean price) and duration-weighted price errors are minimised. The Svensson model fits with a root mean squared error of 3,9 basis points in yield and the spline with 2,8. The curve is upward sloping, from 2,62 % at 0.1 years to 4,70 % at 45 years.

Keywords: term structure of interest rates, Svensson model, cubic splines, zero-coupon rates, government bonds.

Índice

1. Datos y tratamiento **1**

2. Metodología **1**

2.1. Modelo de Svensson 1

2.2. Splines cúbicas 2

3. Resultados **2**

3.1. Svensson 2

3.2. Splines cúbicas y comparación 3

4. Comentario de los resultados **5**

5. Conclusiones **7**

A. Datos y ajuste por título **8**

B. Archivos y código **9**

1. Datos y tratamiento

Muestra. El fichero de cotizaciones contiene 58 bonos y obligaciones; la Obligación 4,000 % 10/2064 no tiene cotización y se excluye, por lo que quedan 57 títulos (2 son cupón cero: el Bono 0,000 % 01/2027 y la Obligación 0,000 % 01/2028). Se usa el **precio ex-cupón medio** $P^{*} = \frac{1}{2}(P^{compra} + P^{venta})$ en porcentaje del nominal. La fecha de contratación es el 11/09/2026 y la fecha valor el 14/09/2026 (dato del propio fichero, D+1 hábil).

Fechas de pago. El fichero indica solo mes y año de vencimiento. Se ha tomado el día habitual de las emisiones del Tesoro según el mes (31/01, 30/03, 30/04, 31/05, 30/07, 31/10) y cupones anuales en el aniversario del vencimiento. Como comprobación, con estas fechas, el precio bruto y la convención del Tema 1 la TIR calculada coincide con la TIR publicada (media compra/venta) con una diferencia media absoluta de 0,3 puntos básicos; la única diferencia relevante (8,4 pb) es la Obligación 1,300 % 10/2026, a mes y medio del vencimiento, donde el redondeo del precio se amplifica en el tipo anual.

Plazos y precio bruto. Según el Tema 1 el cupón corrido es $CC = C \cdot \frac{d}{b}$, con d los días desde el último cupón hasta la fecha valor y b los días entre cupones (ACT/ACT), y el precio bruto es $P = P^{*} + CC$. Los flujos futuros se sitúan en $t_k = \tau + k$ años, con $\tau = \frac{\text{días hasta el próximo cupón}}{b}$ y $k = 0, 1, \ldots$. La TIR y de cada título es la raíz de $P = \sum_k CF_k (1+y)^{-t_k}$ y su duración de Macaulay $D = \sum_k t_k CF_k (1+y)^{-t_k}/P$.

2. Metodología

2.1. Modelo de Svensson

Los tipos al contado con capitalización continua son

$$r_T = \beta_0 + (\beta_1 + \beta_2)\frac{1 - e^{-\kappa_1 T}}{\kappa_1 T} - \beta_2 e^{-\kappa_1 T} + \beta_3 \frac{1 - e^{-\kappa_2 T}}{\kappa_2 T} - \beta_3 e^{-\kappa_2 T}, \tag{2.1}$$

con función de descuento $D(0,T) = e^{-r_T T}$ y forward instantáneo $f(T) = \beta_0 + \beta_1 e^{-\kappa_1 T} + \beta_2 \kappa_1 T e^{-\kappa_1 T} + \beta_3 \kappa_2 T e^{-\kappa_2 T}$. β_0 es el tipo a plazo infinito, $\beta_0 + \beta_1$ el tipo a plazo cero, y β_2, β_3 con κ_1, κ_2 gobiernan dos jorobas. El precio teórico del título j es $\hat{P}_j = \sum_k CF_{jk}\, e^{-r(t_{jk}) t_{jk}}$. Siguiendo el Anexo I, que advierte de que mínimos cuadrados ordinarios sobre precios penaliza demasiado los errores de los títulos largos (un mismo error de TIR produce un

error de precio proporcional a la duración), se estiman los parámetros minimizando

$$\min_{\beta,\kappa} \sum_{j=1}^{n} \Big(\frac{\hat{P}_j - P_j}{D_j} \Big)^2 . \tag{2.2}$$

La optimización (mínimos cuadrados no lineales, algoritmo de región de confianza) parte de los valores del enunciado $(\beta_0, \beta_1, \beta_2, \beta_3, \kappa_1, \kappa_2) = (0{,}05; -0{,}025; 0{,}01; -0{,}04; 1{,}2; 0{,}5)$ y se repite desde otros cinco arranques, con las cotas $\kappa \in [0{,}05; 10]$, $|\beta_{1,2,3}| \leq 0{,}1$ y $\beta_0 \in [0; 0{,}2]$. Se queda el óptimo de menor coste.

2.2. Splines cúbicas

Se aproxima $D(0,T) = 1 + a_1 T + a_2 T^2 + a_3 T^3 + \sum_p b_p (T - \xi_p)_+^3$ (con $D(0,0) = 1$ impuesto). Es una función de clase C^2 y el precio de cada título es lineal en $(a_1, a_2, a_3, b_1, \dots)$, de modo que la estimación es un mínimo cuadrado ponderado (con el mismo peso $1/D_j$) resuelto con álgebra lineal. Nudos internos: $\xi = (1{,}5, 4{,}0, 8{,}0, 15{,}0)$ años, es decir 7 parámetros. Se comprueba que D es positiva y decreciente en $[0; 45{,}1]$ (se cumple).

3. Resultados

3.1. Svensson

Tabla 1: Parámetros estimados (Svensson).

Parámetro	Valor
β_0	0,04930
β_1	-0,02321
β_2	-0,01750
β_3	-0,04232
κ_1	3,04691
κ_2	0,46020

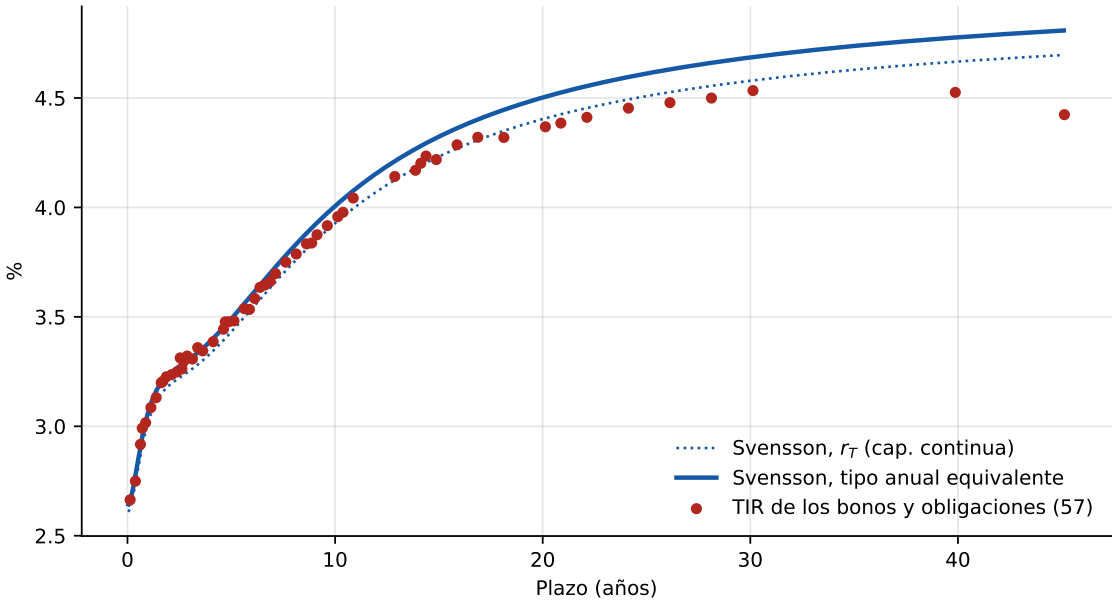


Figura 1: Curva cupón cero de Svensson (línea punteada: tipo r_T con capitalización continua; línea continua: tipo anual equivalente $e^{r_T} - 1$, comparable con la TIR) y TIR de los 57 bonos y obligaciones.

El ajuste (Tabla 2) deja un error medio absoluto de 2,3 pb en TIR; el mayor error es de 23,2 pb (OBL 1,450 10/2071, plazo 45,1 años).

3.2. Splines cúbicas y comparación

Tabla 2: Calidad del ajuste (en la muestra).

	Svensson	Splines
Nº de parámetros	6	7
RMSE precio (€ por 100 nominal)	0,380	0,254
RMSE TIR (pb)	3,92	2,81
Error absoluto medio TIR (pb)	2,31	2,05
Error máximo TIR (pb)	23,20	10,58
Función objetivo $\sum((\hat{P} - P)/D)^2$	0,03082	0,02493
Criterio BIC (sobre residuos ponderados)	-404,5	-412,6

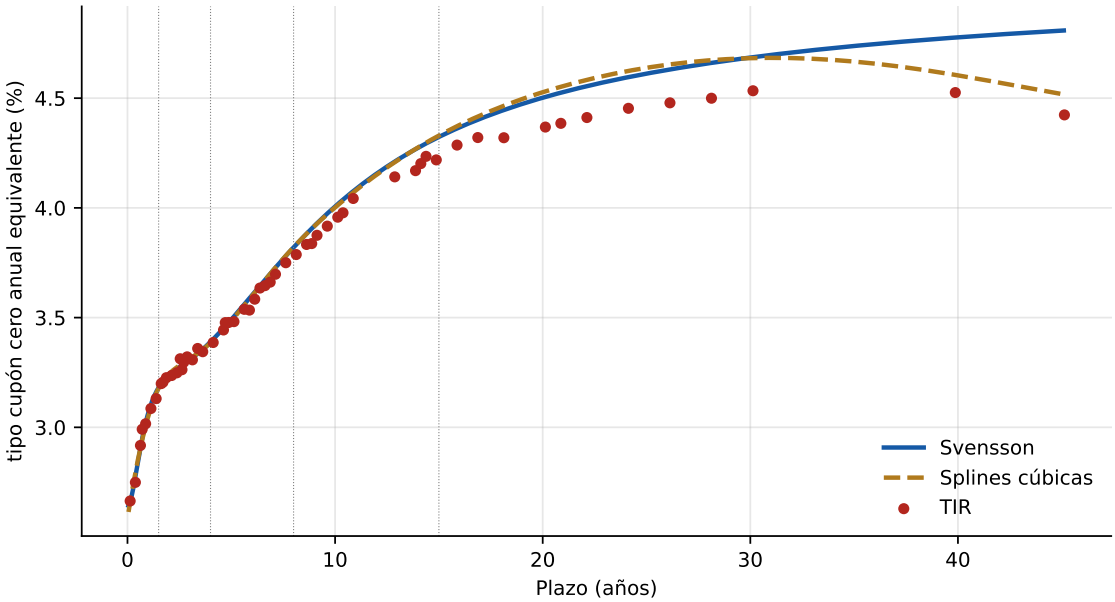


Figura 2: Curvas cupón cero (tipo anual equivalente) de Svensson y de splines (líneas verticales: nudos) y TIR.

Tabla 3: Tipos al contado y forward instantáneos estimados (%). r: capitalización continua; $R = e^r - 1$: anual equivalente.

	Svensson			Splines		
T (años)	r_T	R_T	$f(T)$	r_T	R_T	$f(T)$
0,25	2,672	2,708	2,790	2,690	2,726	2,820
0,50	2,803	2,843	3,069	2,811	2,851	3,038
1	3,020	3,066	3,337	3,005	3,050	3,323
2	3,187	3,238	3,349	3,192	3,244	3,373
3	3,254	3,308	3,459	3,259	3,313	3,437
5	3,431	3,491	3,954	3,425	3,485	3,959
7	3,646	3,714	4,386	3,646	3,714	4,403
10	3,928	4,006	4,735	3,924	4,002	4,707
15	4,233	4,323	4,901	4,239	4,330	4,974
20	4,404	4,502	4,926	4,428	4,527	4,998
30	4,579	4,685	4,930	4,576	4,682	4,650
45	4,696	4,808	4,930	4,420	4,519	3,698

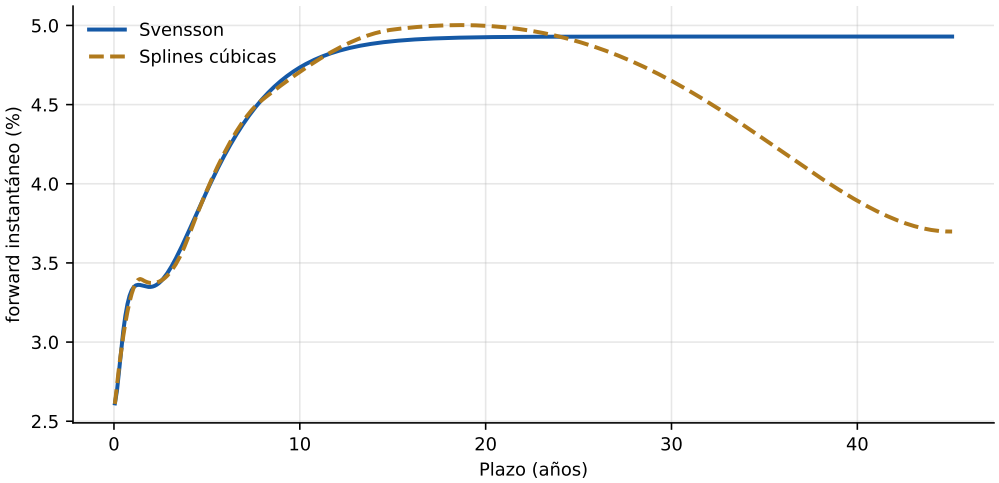


Figura 3: Forward instantáneo implícito.

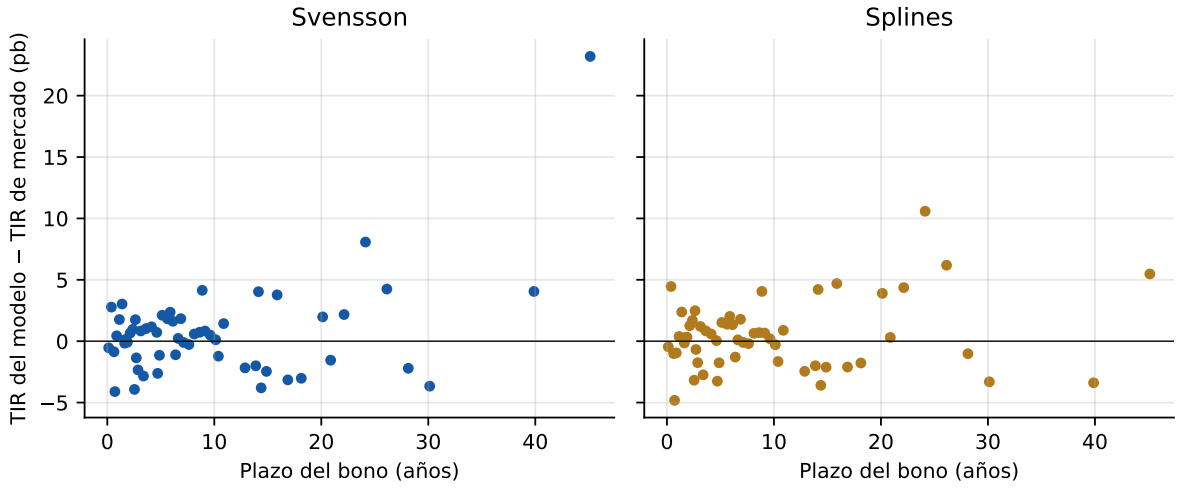


Figura 4: Error del modelo en TIR (modelo menos mercado) según el plazo del bono.

4. Comentario de los resultados

Forma de la curva. La ETTI es *creciente* y cóncava: parte del 2,62 % a 0,1 años, alcanza el 3,02 % al año y se acerca al 4 % a los 10 años y se aproxima al $\beta_0 = 4{,}93$ % (cap. continua) en el tramo largo; en tipo anual equivalente la asíntota es $e^{\beta_0} - 1 = 5{,}05$ %. $\beta_0 + \beta_1 = 2{,}61$ % es el tipo a plazo cero. La pendiente positiva implica, bajo la Teoría Pura de las Expectativas, que se esperan subidas de tipos (forward a 1 año $\approx 3{,}34$ %, a 5 años $\approx 3{,}95$ %); bajo la Preferencia por la Liquidez parte de esa pendiente es prima de plazo y la subida esperada de los tipos sería menor. Es coherente con la subida del Euribor de 2026 observada en el Trabajo 1 (el 12 meses pasa del 2,73 % al 3,30 % entre julio y octubre).

TIR frente a tipo cupón cero. La curva cupón cero y las TIR no coinciden porque la TIR es una media de los tipos al contado ponderada por los flujos del bono (sesgo del cupón): en una curva creciente, los bonos con cupón elevado tienen TIR inferior al cupón cero del mismo vencimiento. En la muestra los puntos se distribuyen en torno a la curva (error absoluto medio de 2,3 pb) con una ligera dispersión por la diferente duración de los cupones y por la liquidez de cada referencia.

Calidad del ajuste. Los errores son pequeños (menos de 4 pb en media cuadrática) pero no nulos: pueden deberse a diferencias de liquidez, a asincronías entre las cotizaciones, a que se trabaja con la media de las posiciones de compra y venta y a la restricción de forma de la función elegida (el Anexo I recuerda que los tipos de la ETTI no son observables). El mayor residuo de Svensson está en el tramo más largo (Obl. 1,450 % 10/2071, 45 años, la TIR del modelo queda por encima de la de mercado), donde hay pocos títulos, poco líquidos.

Svensson frente a splines. La spline ajusta mejor en la muestra (RMSE en TIR de 2,8 frente a 3,9 pb, y también menor BIC con un parámetro más), pero su tramo largo no es fiable: el forward instantáneo baja de más del 5 % a 20 años hasta 3,7 % a 45 años, y el polinomio no está acotado, de modo que extrapolar más allá del último vencimiento carece de sentido (limitación señalada en el Anexo I). Svensson impone una forma suave con forward asintótico constante ($\approx 4{,}93$ %), más estable en el extremo largo y coherente con la teoría, a costa de un ajuste algo peor en el tramo corto y medio.

Robustez e identificación. Los parámetros de Svensson están débilmente identificados: (β_2, κ_1) y (β_3, κ_2) describen formas parecidas y la función objetivo tiene un valle casi plano. La Tabla 4 muestra que seis arranques distintos convergen a tres combinaciones de parámetros muy diferentes con costes casi idénticos (diferencias del orden del 0,3 %), y la Tabla 5 que las curvas resultantes difieren menos de 4 pb en todos los plazos salvo en el de un año con mínimos cuadrados sin ponderar (unos 8 pb). Por tanto, la interpretación económica de cada β debe hacerse con cautela, y solo la curva (no cada parámetro) es robusta.

Tabla 4: Multiarranque de Svensson: valores iniciales y finales ($\beta_0, \beta_1, \beta_2, \beta_3, \kappa_1, \kappa_2$) y coste $\frac{1}{2}\sum e^2$.

#	inicio						final					coste
1	0,0500	-0,0250	0,0100	-0,0400	1,2000	0,5000	0,0492	-0,0243	0,0788	-0,1000	0,7162	0,5599	0,015451
2	0,0400	0,0000	0,0000	0,0000	1,0000	0,3000	0,0492	-0,0243	0,0788	-0,1000	0,7162	0,5599	0,015451
3	0,0450	-0,0200	0,0200	-0,0200	2,0000	0,2000	0,0493	-0,0232	-0,0175	-0,0423	3,0469	0,4602	0,015411
4	0,0500	-0,0300	0,0300	0,0300	0,5000	3,0000	0,0492	-0,0243	-0,1000	0,0847	0,5613	0,7121	0,015442
5	0,0600	-0,0400	-0,0200	0,0200	0,8000	0,1000	0,0492	-0,0243	0,0788	-0,1000	0,7162	0,5599	0,015451
6	0,0500	-0,0200	0,0000	0,0000	2,5000	0,5000	0,0493	-0,0232	-0,0175	-0,0423	3,0469	0,4602	0,015411

Tabla 5: Tipo al contado r_T (%, cap. continua) con distintas especificaciones de la estimación.

Especificación	$T = 1$	$T = 5$	$T = 10$	$T = 30$
Base: ponderación $1/D$, cotas $\lvert\beta\rvert \leq 0{,}1$	3,020	3,431	3,928	4,579
MCO sobre precios (sin ponderar)	3,099	3,425	3,946	4,542
Ponderando por $1/(\text{D_mod} \cdot \text{P})$ ($\approx$ error en TIR)	3,025	3,435	3,928	4,537
Sin cotas en β (solo κ>0)	3,015	3,431	3,929	4,574

Tabla 6: Sensibilidad de la spline a los nudos.

Nudos (años)	Parámetros	$\sum((\hat{P} - P)/D)^2$	RMSE TIR (pb)
2, 5, 10	6	0,03367	3,08
1,5, 4, 8, 15	7	0,02493	2,81
2, 5, 10, 20	7	0,03120	2,84
1, 3, 6, 10, 20	8	0,02144	2,51
2, 5, 10, 20, 30	8	0,03067	2,79

5. Conclusiones

1. La ETTI cupón cero del Tesoro el 11/09/2026 es creciente y cóncava, del 2,6 % a plazos cortos al 4,7 % a 45 años (cap. continua).

2. El modelo de Svensson reproduce las TIR de los 57 títulos con un error cuadrático medio de 3,9 pb, con un tramo largo estable; las splines ajustan algo mejor (2,8 pb) pero extrapolan mal.

3. Los parámetros individuales de Svensson no están bien identificados (valle plano), aunque la curva sí lo está.

4. La diferencia entre TIR y tipos cupón cero refleja el sesgo del cupón y la distinta liquidez de los títulos.

A. Datos y ajuste por título

Título	Cupón	Vencim.	P^*	P bruto	T	D	TIR (%)	TIR Sv.	TIR Spl.
BON 0,000 01/2027	0,00	31/01/2027	98,972	98,972	0,38	0,38	2,749	2,777	2,794
BON 2,350 03/2029	2,35	30/03/2029	97,683	98,764	2,54	2,47	3,312	3,273	3,281
BON 2,400 05/2028	2,40	31/05/2028	98,672	99,369	1,71	1,69	3,206	3,207	3,209
BON 2,500 05/2027	2,50	31/05/2027	99,653	100,379	0,71	0,71	2,991	2,950	2,943
BON 2,600 05/2031	2,60	31/05/2031	96,240	96,995	4,71	4,46	3,477	3,451	3,445
BON 2,700 01/2030	2,70	31/01/2030	97,915	99,587	3,38	3,22	3,359	3,331	3,332
BON 3,500 05/2029	3,50	31/05/2029	100,507	101,524	2,71	2,61	3,297	3,283	3,290
OBL 0,000 01/2028	0,00	31/01/2028	95,833	95,833	1,38	1,38	3,131	3,161	3,155
OBL 0,100 04/2031	0,10	30/04/2031	85,928	85,965	4,62	4,61	3,444	3,451	3,444
OBL 0,500 04/2030	0,50	30/04/2030	90,438	90,625	3,62	3,59	3,345	3,355	3,353
OBL 0,500 10/2031	0,50	31/10/2031	86,213	86,648	5,13	5,05	3,482	3,503	3,497
OBL 0,600 10/2029	0,60	31/10/2029	92,075	92,598	3,13	3,09	3,308	3,316	3,320
OBL 0,700 04/2032	0,70	30/04/2032	85,750	86,013	5,62	5,51	3,538	3,556	3,552
OBL 0,800 07/2027	0,80	30/07/2027	98,115	98,216	0,87	0,87	3,016	3,021	3,007
OBL 0,800 07/2029	0,80	30/07/2029	93,195	93,296	2,87	2,85	3,321	3,298	3,304
OBL 0,850 07/2037	0,85	30/07/2037	72,347	72,455	10,87	10,32	4,043	4,057	4,052
OBL 1,000 07/2042	1,00	30/07/2042	62,710	62,836	15,87	14,34	4,286	4,324	4,333
OBL 1,000 10/2050	1,00	31/10/2050	49,550	50,421	24,13	19,76	4,454	4,534	4,560
OBL 1,200 10/2040	1,20	31/10/2040	68,495	69,540	14,13	12,61	4,202	4,242	4,244
OBL 1,250 10/2030	1,25	31/10/2030	91,893	92,982	4,13	4,00	3,387	3,398	3,392
OBL 1,300 10/2026	1,30	31/10/2026	99,825	100,958	0,13	0,13	2,664	2,659	2,660
OBL 1,400 04/2028	1,40	30/04/2028	97,190	97,715	1,62	1,61	3,199	3,198	3,198
OBL 1,400 07/2028	1,40	30/07/2028	96,727	96,904	1,87	1,86	3,226	3,225	3,230
OBL 1,450 04/2029	1,45	30/04/2029	95,505	96,049	2,62	2,58	3,263	3,280	3,287
OBL 1,450 10/2027	1,45	31/10/2027	98,210	99,473	1,13	1,11	3,086	3,103	3,090
OBL 1,450 10/2071	1,45	31/10/2071	42,305	43,568	45,13	25,12	4,424	4,656	4,478
OBL 1,500 04/2027	1,50	30/04/2027	99,130	99,693	0,62	0,62	2,917	2,909	2,907
OBL 1,850 07/2035	1,85	30/07/2035	85,285	85,518	8,87	8,18	3,837	3,879	3,878
OBL 1,900 10/2052	1,90	31/10/2052	60,745	62,400	26,13	18,49	4,479	4,521	4,541
OBL 2,350 07/2033	2,35	30/07/2033	92,150	92,446	6,87	6,39	3,662	3,680	3,679
OBL 2,550 10/2032	2,55	31/10/2032	94,395	96,617	6,13	5,61	3,584	3,600	3,597
OBL 2,700 10/2048	2,70	31/10/2048	76,120	78,472	22,13	15,57	4,412	4,433	4,455
OBL 2,900 10/2046	2,90	31/10/2046	80,595	83,122	20,13	14,46	4,368	4,388	4,407
OBL 3,000 01/2033	3,00	31/01/2033	96,430	98,288	6,38	5,78	3,635	3,624	3,622
OBL 3,100 07/2031	3,10	30/07/2031	98,328	98,718	4,87	4,58	3,478	3,466	3,460
OBL 3,150 04/2033	3,15	30/04/2033	97,115	98,297	6,62	6,00	3,646	3,648	3,647
OBL 3,150 04/2035	3,15	30/04/2035	95,047	96,230	8,62	7,57	3,833	3,841	3,840
OBL 3,200 10/2035	3,20	31/10/2035	94,885	97,673	9,13	7,80	3,875	3,883	3,882
OBL 3,250 04/2034	3,25	30/04/2034	96,722	97,942	7,62	6,78	3,750	3,747	3,748
OBL 3,300 04/2036	3,30	30/04/2036	95,118	96,356	9,62	8,26	3,917	3,922	3,919
OBL 3,400 10/2036	3,40	31/10/2036	95,412	98,375	10,13	8,44	3,958	3,959	3,955
OBL 3,450 07/2043	3,45	30/07/2043	89,713	90,147	16,87	12,76	4,321	4,289	4,299
OBL 3,450 07/2066	3,45	30/07/2066	80,295	80,730	39,87	20,09	4,526	4,566	4,492
OBL 3,450 10/2034	3,45	31/10/2034	97,670	100,676	8,13	7,00	3,787	3,793	3,794
OBL 3,500 01/2041	3,50	31/01/2041	92,188	94,355	14,38	11,15	4,235	4,197	4,199
OBL 3,550 10/2033	3,55	31/10/2033	99,083	102,175	7,13	6,23	3,698	3,697	3,697
OBL 3,900 07/2039	3,90	30/07/2039	97,620	98,112	12,87	10,28	4,141	4,120	4,117
OBL 3,950 10/2056	3,95	31/10/2056	90,500	93,941	30,13	16,85	4,534	4,497	4,501
OBL 4,000 10/2054	4,00	31/10/2054	92,105	95,590	28,13	16,29	4,500	4,478	4,490
OBL 4,200 01/2037	4,20	31/01/2037	101,843	104,443	10,38	8,43	3,978	3,965	3,961
OBL 4,250 07/2047	4,25	30/07/2047	98,165	98,701	20,87	14,10	4,385	4,370	4,388
OBL 4,700 07/2041	4,70	30/07/2041	105,227	105,820	14,87	11,08	4,219	4,194	4,198

Título	Cupón	Vencim.	P^*	P bruto	T	D	TIR (%)	TIR Sv.	TIR Spl.
OBL 4,900 07/2040	4,90	30/07/2040	107,567	108,185	13,87	10,47	4,170	4,150	4,150
OBL 5,150 10/2028	5,15	31/10/2028	103,868	108,354	2,13	1,99	3,236	3,243	3,249
OBL 5,150 10/2044	5,15	31/10/2044	110,280	114,767	18,13	12,02	4,320	4,289	4,302
OBL 5,750 07/2032	5,75	30/07/2032	111,560	112,285	5,87	5,16	3,534	3,558	3,554
OBL 6,000 01/2029	6,00	31/01/2029	106,185	109,900	2,38	2,22	3,248	3,258	3,265

B. Archivos y código

El libro `Trabajo2_ETTI_RentaFija.xlsx` reproduce todo el cálculo con fórmulas (hojas *Parametros*, *Bonos*, *Flujos*, *Splines*, *Curva*); en *Parametros* está preparada la función objetivo de Svensson para Solver y los coeficientes de las splines se obtienen con `LINEST`.

t2core.py

```
# -*- coding: utf-8 -*-
"""Nucleo de calculo del Trabajo 2: datos, flujos, precios y estimacion de la ETTI (Svensson y splines cubicas)."""
import re, datetime as dt, glob
import numpy as np
import openpyxl
from scipy.optimize import brentq, least_squares

LIQ = dt.date(2026, 9, 14)          # fecha valor (celda R4 del Excel); contratacion 11/09/2026
RAIZ = r"C:\Users\luzur\OneDrive - Universidad de Alcala\Documentos\UNIVERSIDAD\MUCAF\SEGUNDO ANO\Renta Fija\Tema 3"
# Dia de vencimiento del Tesoro segun el mes (calendario de emisiones del Tesoro): 31/01, 30/03, 30/04, 31/05, 30/07, 31/10, 30/11
DIA = {1: 31, 3: 30, 4: 30, 5: 31, 7: 30, 10: 31, 11: 30}

def leer():
    f = glob.glob(RAIZ + r"\COTIZ*.xlsx")[0]
    ws = openpyxl.load_workbook(f, data_only=True).active
    bonos = []
    for r in range(6, ws.max_row + 1):
        nombre = ws.cell(r, 1).value
        if not nombre or not str(nombre).startswith(("BON", "OBL")): continue
        pc, pv = ws.cell(r, 6).value, ws.cell(r, 7).value
        if not isinstance(pc, (int, float)): continue            # sin cotizacion
        m = re.match(r"(BON|OBL) TESORO - ([\d,]+) (\d\d)/(\d{4})", nombre)
        cup = float(m.group(2).replace(",", "."))
        mes, anio = int(m.group(3)), int(m.group(4))
        venc = dt.date(anio, mes, DIA[mes])
        bonos.append(dict(fila=r, nombre=nombre, isin=ws.cell(r, 2).value, tipo=m.group(1), cupon=cup, venc=venc,
                          p_c=pc, p_v=pv, tir_c=ws.cell(r, 5).value, tir_v=ws.cell(r, 8).value,
                          imp_c=ws.cell(r, 4).value, imp_v=ws.cell(r, 9).value))
    return bonos

```

```
31 def anios_atras(d, k=1):
32     return d.replace(year=d.year - k)
33
34 def flujos(b, liq=LIQ):
35     """Fechas, importes y plazos (anos) de los flujos futuros. Plazo = fraccion del
           primer periodo + k (convencion del Tema 1)."""
36     v = b["venc"]
37     fechas = []
38     d = v
39     while d > liq:
40         fechas.append(d); d = anios_atras(d)
41     fechas = fechas[::-1]
42     prev = anios_atras(fechas[0])
43     base = (fechas[0] - prev).days
44     frac = (fechas[0] - liq).days / base
45     t = np.array([frac + k for k in range(len(fechas))])
46     cf = np.full(len(fechas), b["cupon"]); cf[-1] += 100.0
47     cc = b["cupon"] * (liq - prev).days / base
48     return dict(fechas=fechas, t=t, cf=cf, cc=cc, prev=prev, base=base)
49
50 def precio_tir(cf, t, y):
51     return float(np.sum(cf / (1 + y) ** t))
52
53 def tir(cf, t, P):
54     return brentq(lambda y: precio_tir(cf, t, y) - P, -0.5, 1.0)
55
56 def duracion(cf, t, y):
57     pv = cf / (1 + y) ** t
58     return float((t * pv).sum() / pv.sum())
59
60 def preparar():
61     bonos = leer()
62     for b in bonos:
63         fl = flujos(b); b.update(fl)
64         b["P_m"] = (b["p_c"] + b["p_v"]) / 2                    # precio ex-cupon medio
               compra/venta
65         b["P_b"] = b["P_m"] + b["cc"]                           # precio bruto
66         b["y"] = tir(b["cf"], b["t"], b["P_b"])
67         b["y_pub"] = (b["tir_c"] + b["tir_v"]) / 2 / 100
68         b["D"] = duracion(b["cf"], b["t"], b["y"])
69         b["T"] = float(b["t"][-1])
70     return bonos
71
72 # --------------------------- Svensson ---------------------------------------------
73 def svensson(T, p):
74     b0, b1, b2, b3, k1, k2 = p
75     T = np.asarray(T, float)
76     a1 = (1 - np.exp(-k1 * T)) / (k1 * T); a2 = (1 - np.exp(-k2 * T)) / (k2 * T)
77     return b0 + (b1 + b2) * a1 - b2 * np.exp(-k1 * T) + b3 * a2 - b3 * np.exp(-k2 * T)
78
79 def svensson_fwd(T, p):
80     b0, b1, b2, b3, k1, k2 = p
81     T = np.asarray(T, float)
82     return b0 + b1 * np.exp(-k1 * T) + b2 * k1 * T * np.exp(-k1 * T) + b3 * k2 * T * np.
           exp(-k2 * T)
83
84 def precio_modelo(bonos, p):
```

```
    return np.array([np.sum(b["cf"] * np.exp(-svensson(b["t"], p) * b["t"])) for b in
        bonos])

def ajustar_svensson(bonos, p0=(0.05, -0.025, 0.01, -0.04, 1.2, 0.5), pesos="dur"):
    P = np.array([b["P_b"] for b in bonos]); D = np.array([b["D"] for b in bonos])
    w = {"dur": 1 / D, "mco": np.ones_like(D)}[pesos]
    lb = [0, -0.2, -0.2, -0.2, 0.05, 0.05]; ub = [0.2, 0.2, 0.2, 0.2, 10, 10]
    fun = lambda p: w * (precio_modelo(bonos, p) - P)
    s = least_squares(fun, p0, bounds=(lb, ub), x_scale=[0.05, 0.05, 0.05, 0.05, 1, 1],
        xtol=1e-14, ftol=1e-14, gtol=1e-14, max_nfev=5000)
    return s

# ---------------------------- Splines cubicas sobre D(t)
    --------------------------------
def base_spline(t, nudos):
    """Funciones␣base␣con␣a0␣=␣1␣(D(0)=1):␣[t,␣t2,␣t3,␣(t-_p)^3_+␣...].␣D(t)␣=␣1␣+␣␣_j␣_j
        (t)."""
    t = np.asarray(t, float)
    cols = [t, t ** 2, t ** 3] + [np.maximum(t - x, 0) ** 3 for x in nudos]
    return np.vstack(cols)

def matriz_spline(bonos, nudos):
    A = np.zeros((len(bonos), 3 + len(nudos)))
    for i, b in enumerate(bonos):
        A[i] = (b["cf"][None, :] * base_spline(b["t"], nudos)).sum(axis=1)
    return A

def ajustar_spline(bonos, nudos, pesos="dur"):
    P = np.array([b["P_b"] for b in bonos]); D = np.array([b["D"] for b in bonos])
    A = matriz_spline(bonos, nudos)
    c0 = np.array([b["cf"].sum() for b in bonos])          # D(t)=1 + ...  -> P -  CF = A
    w = 1 / D if pesos == "dur" else np.ones_like(D)
    beta, *_ = np.linalg.lstsq(A * w[:, None], (P - c0) * w, rcond=None)
    return beta

def D_spline(t, beta, nudos):
    t = np.asarray(t, float)
    return (1 + beta @ base_spline(t.ravel(), nudos)).reshape(t.shape)

def tir_modelo(bonos, Pmod):
    return np.array([tir(b["cf"], b["t"], p) for b, p in zip(bonos, Pmod)])
```

t2.py

```
# -*- coding: utf-8 -*-
"""Trabajo␣2␣␣Estimacion␣de␣la␣ETTI␣cupon␣cero␣(Svensson␣y␣splines␣cubicas)␣con␣las␣
    cotizaciones␣del␣11-sep-2026."""
import json, sys, datetime as dt
from pathlib import Path
import numpy as np
from scipy.optimize import least_squares
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
sys.path.insert(0, str(Path(__file__).parent))
```

```
from t2core import *

OUT = Path(__file__).parent
B = preparar()
P = np.array([b["P_b"] for b in B]); Dur = np.array([b["D"] for b in B]); yM = np.array([b["y"] for b in B]); Tm = np.array([b["T"] for b in B])

# ---------------------------------- Svensson ----------------------------------------------
P0 = (0.05, -0.025, 0.01, -0.04, 1.2, 0.5)                    # valores iniciales del enunciado
LB = [0, -0.1, -0.1, -0.1, 0.05, 0.05]; UB = [0.2, 0.1, 0.1, 0.1, 10, 10]
def fit_sv(w, p0, lb=LB, ub=UB):
    f = lambda p: w * (precio_modelo(B, p) - P)
    return least_squares(f, p0, bounds=(lb, ub), x_scale=[.05, .05, .05, .05, 1, 1], xtol=1e-14, ftol=1e-14, gtol=1e-14, max_nfev=5000)
w_dur = 1 / Dur
starts = [P0, (0.04, 0, 0, 0, 1, 0.3), (0.045, -0.02, 0.02, -0.02, 2, 0.2), (0.05, -0.03, 0.03, 0.03, 0.5, 3), (0.06, -0.04, -0.02, 0.02, 0.8, 0.1), (0.05, -0.02, 0.0, 0.0, 2.5, 0.5)]
runs = [fit_sv(w_dur, s) for s in starts]
best = min(runs, key=lambda r: r.cost)
psv = best.x
Pm_sv = precio_modelo(B, psv); y_sv = tir_modelo(B, Pm_sv)
# alternativas de robustez
alt = {}
alt["MCO sobre precios (sin ponderar)"] = fit_sv(np.ones_like(Dur), psv).x
w_y = 1 / (Dur / (1 + yM) * P)                                # error de precio / (D_modP)  error en TIR
alt["Ponderando por 1/(D_modP) ( error en TIR)"] = fit_sv(w_y, psv).x
alt["Sin cotas en  (solo >0)"] = fit_sv(w_dur, P0, [-1, -1, -1, -1, 0.01, 0.01], [1, 1, 1, 1, 50, 50]).x

# ---------------------------------- Splines -----------------------------------------------
NUDOS = (1.5, 4.0, 8.0, 15.0)
beta = ajustar_spline(B, NUDOS)
A = matriz_spline(B, NUDOS); c0 = np.array([b["cf"].sum() for b in B])
Pm_sp = c0 + A @ beta; y_sp = tir_modelo(B, Pm_sp)
cfg = {}
for nud in [(2, 5, 10), (1.5, 4, 8, 15), (2, 5, 10, 20), (1, 3, 6, 10, 20), (2, 5, 10, 20, 30)]:
    bt = ajustar_spline(B, nud); Pm = c0 + matriz_spline(B, nud) @ bt
    cfg[str(nud)] = dict(nudos=nud, k=len(nud) + 3, sse=float(np.sum(((Pm - P) / Dur) ** 2)), rmse_tir=float(1e4 * np.sqrt(np.mean((tir_modelo(B, Pm) - yM) ** 2))))

def metr(Pm, y, k):
    n = len(B); e = (Pm - P) / Dur
    sse = float(np.sum(e ** 2))
    return dict(k=k, rmse_precio=float(np.sqrt(np.mean((Pm - P) ** 2))), mae_precio=float(np.mean(abs(Pm - P))),
                rmse_tir_bp=float(1e4 * np.sqrt(np.mean((y - yM) ** 2))), mae_tir_bp=float(1e4 * np.mean(abs(y - yM))),
                max_tir_bp=float(1e4 * np.max(abs(y - yM))), sse_pond=sse, bic=float(n * np.log(sse / n) + k * np.log(n)))
m_sv = metr(Pm_sv, y_sv, 6); m_sp = metr(Pm_sp, y_sp, len(NUDOS) + 3)

```

```
# ---------------------------------- malla de resultados ----------------------------------
grid = np.array([0.1, 0.25, 0.5, 1, 2, 3, 4, 5, 7, 10, 15, 20, 25, 30, 40, 45])
tg = np.linspace(0.05, 45.1, 400)
r_sv = svensson(grid, psv); f_sv = svensson_fwd(grid, psv)
r_sp = -np.log(D_spline(grid, beta, NUDOS)) / grid
# forward instantaneo de la spline: f = -D'/D
def dD(t): t = np.asarray(t, float); return beta[0] + 2 * beta[1] * t + 3 * beta[2] * t ** 2 + sum(3 * beta[3 + j] * np.maximum(t - x, 0) ** 2 for j, x in enumerate(NUDOS))
f_sp = -dD(grid) / D_spline(grid, beta, NUDOS)
Dt = D_spline(tg, beta, NUDOS)
res = dict(
    liq=LIQ.isoformat(), n=len(B), sv=dict(params=dict(zip(["beta0", "beta1", "beta2", "beta3", "kappa1", "kappa2"], psv.tolist()))), cost=float(best.cost), metr=m_sv,
                                        arranques=[dict(inicio=list(map(float, s)),
                                            final=list(map(float, r.x)), coste=float(r.cost)) for s, r in zip(starts, runs)]),
    sp=dict(nudos=list(NUDOS), beta=beta.tolist(), metr=m_sp, monotona=bool(np.all(np.diff(Dt) <= 1e-12)), positiva=bool(Dt.min() > 0), config=cfg),
    alt={k: (v.tolist() if v is not None else None) for k, v in alt.items()},
    grid=grid.tolist(), r_sv=(r_sv * 100).tolist(), f_sv=(f_sv * 100).tolist(), r_sp=(r_sp * 100).tolist(), f_sp=(f_sp * 100).tolist(),
    bonos=[dict(nombre=b["nombre"], isin=b["isin"], cupon=b["cupon"], venc=b["venc"].isoformat(), P_m=b["P_m"], cc=b["cc"], P_b=b["P_b"], y=b["y"] * 100, y_pub=b["y_pub"] * 100,
            D=b["D"], T=b["T"], P_sv=float(Pm_sv[i]), y_sv=float(y_sv[i] * 100), P_sp=float(Pm_sp[i]), y_sp=float(y_sp[i] * 100)) for i, b in enumerate(B)])
(OUT / "t2_resultados.json").write_text(json.dumps(res, indent=1, ensure_ascii=False), encoding="utf-8")

# ---------------------------------- graficos ----------------------------------------------
plt.rcParams.update({"font.size": 9, "axes.grid": True, "grid.alpha": .3, "axes.spines.top": False, "axes.spines.right": False})
C = {"sv": "#1559a6", "sp": "#b07a1d", "tir": "#b3261e"}
eff = lambda r: np.exp(r) - 1
fig, ax = plt.subplots(figsize=(7.2, 4))
ax.plot(tg, svensson(tg, psv) * 100, ":", color=C["sv"], lw=1.2, label=r"Svensson␣$r_T$␣(cap.␣continua)")
ax.plot(tg, eff(svensson(tg, psv)) * 100, "-", color=C["sv"], lw=2, label="Svensson␣tipo␣anual␣equivalente")
ax.scatter(Tm, yM * 100, s=16, color=C["tir"], zorder=3, label="TIR␣de␣los␣bonos␣y␣obligaciones␣(57)")
ax.set_xlabel("Plazo␣(anos)"); ax.set_ylabel("%"); ax.legend(frameon=False, loc="lower␣right")
fig.tight_layout(); fig.savefig(OUT / "fig_svensson.pdf"); plt.close(fig)

fig, ax = plt.subplots(figsize=(7.2, 4))
ax.plot(tg, eff(svensson(tg, psv)) * 100, color=C["sv"], lw=2, label="Svensson")
ax.plot(tg, eff(-np.log(np.where(Dt > 0, Dt, np.nan)) / tg) * 100, "--", color=C["sp"], lw=2, label="Splines␣cubicas")
ax.scatter(Tm, yM * 100, s=16, color=C["tir"], zorder=3, label="TIR")
for x in NUDOS: ax.axvline(x, color="grey", lw=.5, ls=":")
ax.set_xlabel("Plazo␣(anos)"); ax.set_ylabel("tipo␣cupon␣cero␣anual␣equivalente␣(%)"); ax.legend(frameon=False, loc="lower␣right")
fig.tight_layout(); fig.savefig(OUT / "fig_comparacion.pdf"); plt.close(fig)
```

```
91
92  fig, ax = plt.subplots(figsize=(7.2, 3.6))
93  ax.plot(tg, svensson_fwd(tg, psv) * 100, color=C["sv"], lw=2, label="Svensson")
94  ax.plot(tg, -dD(tg) / Dt * 100, "--", color=C["sp"], lw=2, label="Splines␣cubicas")
95  ax.set_xlabel("Plazo␣(anos)"); ax.set_ylabel("forward␣instantaneo␣(%)"); ax.legend(
        frameon=False)
96  fig.tight_layout(); fig.savefig(OUT / "fig_forward.pdf"); plt.close(fig)
97
98  fig, axs = plt.subplots(1, 2, figsize=(7.4, 3.2), sharey=True)
99  for ax, y, nm, c in zip(axs, (y_sv, y_sp), ("Svensson", "Splines"), (C["sv"], C["sp"])):
100     ax.axhline(0, color="k", lw=.6); ax.scatter(Tm, (y - yM) * 1e4, s=14, color=c); ax.
            set_title(nm); ax.set_xlabel("Plazo␣del␣bono␣(anos)")
101 axs[0].set_ylabel("TIR␣del␣modelo␣␣TIR␣de␣mercado␣(pb)")
102 fig.tight_layout(); fig.savefig(OUT / "fig_residuos.pdf"); plt.close(fig)
103
104 fig, ax = plt.subplots(figsize=(7.2, 3.4))
105 ax.plot(tg, np.exp(-svensson(tg, psv) * tg), color=C["sv"], lw=2, label="Svensson"); ax.
        plot(tg, Dt, "--", color=C["sp"], lw=2, label="Splines")
106 ax.set_xlabel("T␣(anos)"); ax.set_ylabel("D(0,T)"); ax.legend(frameon=False)
107 fig.tight_layout(); fig.savefig(OUT / "fig_descuento.pdf"); plt.close(fig)
108
109 # ---------------------------------- salida en consola
        ----------------------------------------
110 print("Svensson", np.round(psv, 5), "coste", best.cost)
111 for s, r in zip(starts, runs): print("␣␣inicio", np.round(s, 3), "->", np.round(r.x, 4),
        round(r.cost, 6))
112 print("Metricas␣Svensson", {k: round(v, 3) for k, v in m_sv.items()}); print("Metricas␣
        Spline", {k: round(v, 3) for k, v in m_sp.items()})
113 print("alt", {k: (np.round(v, 4).tolist() if v is not None else None) for k, v in alt.
        items()})
114 print("grid", grid.tolist()); print("r_sv", np.round(r_sv * 100, 3).tolist()); print("
        r_sp", np.round(r_sp * 100, 3).tolist())
115 print("f_sv", np.round(f_sv * 100, 3).tolist()); print("f_sp", np.round(f_sp * 100, 3).
        tolist())
116 print("spline␣mono/pos", res["sp"]["monotona"], res["sp"]["positiva"])
```